

DO-178C

SOFTWARE CODE STANDARDS PLAN

Avionics Software Certification Document

Document Number	DOC-SCS-001
Revision	A (Initial Release)
Applicable Standard	DO-178C (RTCA, 2011)
DAL Applicability	DAL A / DAL B / DAL C
Date	May 2026

Prepared in accordance with RTCA DO-178C Section 11.1 – Software Development Plan Requirements

1. PURPOSE AND SCOPE

This Software Code Standards Plan (SCSP) defines the programming languages, methods, rules, and tools to be used for coding avionics software in compliance with RTCA DO-178C. It is a mandatory planning document required under DO-178C Section 11.1 (Software Development Plan) and must be reviewed during Stage of Involvement (SOI) audits by the DER/ACO.

In simple terms: this document is the rulebook that every software developer on the project must follow when writing code. Think of it like the building codes a construction crew must follow — without them, the building (or airplane software) might not be safe.

NOTE *DO-178C Section 11.1 states: 'The Software Development Plan shall describe... the software code standards.' This document fulfills that requirement and must be baselined and placed under configuration management before coding begins.*

Field	Details
Document Type	Software Code Standards Plan (SCSP)
DO-178C Reference	Section 11.1, Table A-1 (Planning Objectives)
Scope	All software components subject to DO-178C certification
Baseline Required	Yes — prior to start of software coding activities
SOI Review	SOI-1 (Planning Audit) and SOI-2 (Development Audit)

2. WHAT ARE SOFTWARE CODE STANDARDS?

Software Code Standards define the set of rules that govern HOW software is written. They don't just say what the software should do – they define HOW it must be written, structured, and documented. In avionics, poor coding practices can introduce subtle bugs that may only appear under rare flight conditions, making code standards a critical safety layer.

DO-178C identifies five major categories of code standards that every project must address. These are sections (a) through (e) in the standard. The remainder of this document covers each one in depth, with avionics-specific examples.

Ref	Category	Description
(a)	Programming Language Standards	Rules for which language(s) to use and which features are permitted
(b)	Source Code Presentation Standards	Rules for how code looks: formatting, line length, indentation, documentation
(c)	Naming Conventions	Rules for naming variables, functions, files, and constants consistently
(d)	Coding Conditions & Constraints	Rules limiting complexity, coupling between modules, and expression complexity
(e)	Coding Constraints Tool	Rules governing which tools are approved for coding and how they are used

3. SECTION (a) – PROGRAMMING LANGUAGE STANDARDS

This is about deciding: WHICH programming language(s) will we use to write the software, and WHICH parts/features of that language are we allowed to use?

Not all features of a programming language are safe for avionics use. DO-178C requires you to either use a defined subset of the language, or use the full language but justify every unsafe feature. The goal is to ensure the behavior of the code is always predictable and understandable.

3.1 Why Language Choice Matters in Aviation

In avionics software, unexpected behavior is catastrophic. Some programming language features produce behavior that:

- Is defined by the hardware (not the language standard), so it may differ on different processors
- Depends on compiler implementation choices that vary between tool vendors
- Can produce different results depending on how the code is compiled (debug vs. release builds)
- Is simply too complex for a human reviewer to verify reliably

NOTE *DO-178C Section 11.1 requires the SCSP to identify the programming language(s) and, for each language, 'reference the data that unambiguously defines the syntax, the control behavior, the data behavior, and the side-effects of the language.'*

3.2 Approved Languages for This Project

The following languages are approved for use. Each is governed by a formal language standard that is referenced as the definitive definition of language behavior.

Language	Governing Standard	Approved Subset / Restriction
C (primary)	ISO/IEC 9899:1990 (C90) / MISRA C:2012	MISRA C:2012 compliant subset; all 143 rules enforced
Ada (legacy)	ISO/IEC 8652:2012 (Ada 2012)	Restricted: tasking & exceptions only with DER approval
Assembly	Processor Reference Manual (target CPU)	Only for hardware initialization routines; code-reviewed

3.3 Prohibited Language Features

The following C language features are PROHIBITED because they introduce undefined, implementation-defined, or unspecified behavior that cannot be reliably verified:

- Dynamic memory allocation (malloc, calloc, free, realloc) – Memory must be pre-allocated statically at startup
- Recursion – Stack depth becomes unpredictable; prohibited per MISRA C:2012 Rule 17.2
- Goto statements – Prohibited per MISRA C:2012 Rule 15.1; makes control flow unanalyzable
- setjmp / longjmp – Causes non-local jumps; violates structured control flow
- Variable-length arrays (VLAs) – Stack usage becomes runtime-dependent; stack overflow risk
- Undefined behavior from signed integer overflow – Must use explicit range checks
- Bit-field types other than unsigned int or int – Behavior is implementation-defined
- Multiple assignments in a single expression (e.g., a = b = c++) – Sequence point issues

AVIONICS EXAMPLE: PROHIBITED vs. PERMITTED C Code for FMS Altitude Buffer

```
// PROHIBITED: Dynamic memory allocation
float* alt_buffer = (float*)malloc(100 * sizeof(float)); // BANNED

// PROHIBITED: Recursion in descent calculation
float calc_descent(int step) { // BANNED
    return calc_descent(step - 1) + ...;
}

// PERMITTED: Static pre-allocation (MISRA compliant)
#define ALT_BUFFER_SIZE 100U
static float alt_buffer[ALT_BUFFER_SIZE]; // OK

// PERMITTED: Iterative loop (no recursion)
float calc_descent(void) { // OK
    float result = 0.0F;
    uint8_t i;
    for (i = 0U; i < DESCENT_STEPS; i++) {
        result += step_delta[i];
    }
    return result;
}
```

4. SECTION (b) — SOURCE CODE PRESENTATION STANDARDS

Source Code Presentation Standards define HOW the code must be visually formatted and documented. This is not about aesthetics — it is a safety requirement. Consistent formatting allows human reviewers to quickly spot errors, understand the code structure, and verify it against design documents during DO-178C audits.

NOTE *DO-178C requires that code be reviewable by humans (not just machines). A reviewer who must spend hours decoding poorly formatted code is more likely to miss a safety-critical defect.*

4.1 Line Length Restriction

Maximum line length: 80 characters. This standard dates to physical printout reviews but remains important today: long lines wrap on displays, are difficult to diff in configuration management tools, and often indicate code that should be refactored into smaller, clearer expressions.

AVIONICS EXAMPLE: Line Length — AHRS Attitude Calculation

```
// NON-COMPLIANT: 103 characters (too long, wraps in review tools)
roll_angle_deg = atan2f(2.0F*(q0*q1 + q2*q3), 1.0F - 2.0F*(q1*q1 + q2*q2)) * RAD_TO_DEG;

// COMPLIANT: Split to <= 80 characters per line
float quat_roll_term1 = 2.0F * (q0*q1 + q2*q3);
float quat_roll_term2 = 1.0F - 2.0F*(q1*q1 + q2*q2);
roll_angle_deg = atan2f(quat_roll_term1, quat_roll_term2)
    * RAD_TO_DEG;
```

4.2 Indentation Standard

All code shall use 4-space indentation. Tabs are PROHIBITED because tab-width rendering differs between editors and code review tools. Consistent indentation is required for DO-178C structural coverage analysis tools to parse code correctly.

AVIONICS EXAMPLE: Indentation — Warning Flag Logic (Flight Warning Computer)

```
// NON-COMPLIANT: Mixed tabs and spaces
if (ias_knots > V_MO) {
    set_warning(OVERSPEED); /* TAB — PROHIBITED */
    log_event(EVT_OVERSPEED); /* 4 spaces */
}

// COMPLIANT: 4-space indentation throughout
if (ias_knots > V_MO) {
    set_warning(OVERSPEED); /* 4 spaces */
}
```

```
log_event(EVT_OVERSPEED); /* 4 spaces */  
}
```

4.3 Blank Line Usage

- One blank line between logically related code blocks within a function
- Two blank lines between function definitions
- No trailing whitespace on any line
- Files must end with a single newline character

4.4 Source Code Documentation Standards

Every source file, every function, and every non-obvious variable must be documented. Documentation enables traceability — a DER must be able to trace from a requirement, through design, into the code, and back. Without clear documentation this trace is broken.

4.4.1 File Header (MANDATORY): Every .c and .h file shall begin with the following block comment:

AVIONICS EXAMPLE: Mandatory File Header — TCAS Resolution Advisory Module

```
/*=====
 * FILE:      tcas_ra.c
 * COMPONENT: Traffic Collision Avoidance System (TCAS II)
 * PURPOSE:   Computes Resolution Advisory (RA) maneuver
 *            commands per RTCA DO-185B logic.
 * AUTHOR:    J. Smith
 * DATE:      2026-05-01
 * REVISION:  1.3
 * DAL:       DAL B
 * REFERENCES: SRD-TCAS-042, TCAS_LLD_003
 * INPUTS:    Own-ship altitude, intruder altitude, closure rate
 * OUTPUTS:   ra_command_t (CLIMB / DESCEND / MAINTAIN)
 * AFFECTED:  Global: g_ra_active_flag
 *=====*/
```

4.4.2 Function Header (MANDATORY for every function): Each function shall be preceded by:

- Purpose: what the function does
- Parameters: each input and output, with units and valid ranges
- Return value: meaning, units, range
- Requirements traced: SRD/LLD IDs
- Side effects: any global data modified

AVIONICS EXAMPLE: Function Header — Autopilot Altitude Hold

```
/*-----  
* FUNCTION: ap_compute_alt_error  
* PURPOSE: Computes altitude error for altitude hold mode.  
*         Returns the signed deviation of current pressure  
*         altitude from target altitude.  
* INPUTS:  
*   target_ft - Target altitude in feet MSL [0..51000]  
*   current_ft - Current pressure altitude in feet MSL  
* RETURNS:  
*   float - Altitude error in feet [-999..+999]  
*         Positive = aircraft is BELOW target  
*         Negative = aircraft is ABOVE target  
* REQUIREMENTS: AP-SRD-017, AP-LLD-031  
* SIDE EFFECTS: None  
*-----*/  
float ap_compute_alt_error(float target_ft, float current_ft)  
{  
    return (target_ft - current_ft);  
}
```


5. SECTION (c) – NAMING CONVENTIONS

Naming conventions define exactly how software entities – functions, variables, constants, files, and modules – must be named. In avionics certification, consistent naming is not just a style preference: it enables automated traceability, structural coverage analysis, and code review without ambiguity.

Think of it like the NATO phonetic alphabet. Every pilot worldwide uses 'Alpha, Bravo, Charlie' instead of 'A, B, C.' The consistency eliminates confusion under pressure. Similarly, a consistent naming scheme eliminates confusion during code review, testing, and DO-178C audit.

NOTE *Poor naming is a defect category tracked in DO-178C code reviews. A name like 'x' or 'temp' gives reviewers no information about the purpose, units, or valid range of a value – making requirement traceability impossible.*

5.1 Component and Subprogram Naming

Module (component) names shall use UPPER_SNAKE_CASE prefixes that reflect the avionics system. Function names shall use lower_snake_case with the module prefix. This creates a namespace in C that prevents name collisions across subsystems.

AVIONICS EXAMPLE: Component / Function Naming

// PATTERN: <SYSTEM>_<SUBSYSTEM>_<verb>_<noun>

// Flight Management System – Route computation

void fms_nav_compute_great_circle(void);

float fms_nav_get_cross_track_error(void);

bool fms_nav_is_waypoint_sequenced(void);

// Autopilot – Lateral mode

void ap_lat_engage_heading_hold(float hdg_deg);

float ap_lat_compute_bank_command(void);

// TCAS – Resolution Advisory

ra_cmd_t tcas_ra_compute_advisory(void);

void tcas_ra_cancel_advisory(void);

// NON-COMPLIANT examples (BANNED):

void compute(void); // No system prefix – ambiguous

float f1(float x); // Meaningless name

int TempCalc(); // Mixed case, no prefix

5.2 Variable Naming Conventions

All variable names shall follow the Hungarian-inspired type-prefixed scheme below. Units shall be

appended as suffixes where the physical unit matters for safety.

Type Prefix	Data Type	Example (Avionics)
b_	bool (Boolean)	b_gear_down, b_autopilot_engaged, b_tcas_ra_active
u8_	uint8_t	u8_flap_position_deg, u8_engine_count
u16_	uint16_t	u16_radio_alt_ft, u16_ias_knots
u32_	uint32_t	u32_gps_timestamp_ms, u32_frame_counter
i16_	int16_t	i16_pitch_deg, i16_roll_deg (signed angles)
f32_	float (32-bit)	f32_airspeed_kts, f32_alt_error_ft
e_	enum type	e_flight_phase, e_ap_lateral_mode
g_	global variable	g_b_master_warning, g_f32_current_alt_ft
k_	constant (file-local)	k_u16_max_alt_ft = 51000U

5.3 Constant Naming

All constants (defined with `#define` or `const`) shall use UPPER_SNAKE_CASE with the module prefix and a unit suffix where applicable. This makes it immediately obvious that a value is a constant (not a variable) and what units it is in.

AVIONICS EXAMPLE: Constants — Aircraft Performance Limits

```
/* Speed limits — units appended as suffix */
#define FMS_VMO_KIAS      (340U) /* Max operating speed, knots IAS */
#define FMS_MMO_MACH100THS (82U) /* Max Mach * 100 (= Mach 0.82) */
#define AP_MAX_BANK_DEG   (30U) /* Autopilot max bank angle, deg */
#define TCAS_THREAT_TAU_SEC (35U) /* TCAS threat threshold, seconds */
#define ADC_BARO_REF_HPA   (1013U) /* Standard baro reference, hPa */

/* NON-COMPLIANT examples (BANNED): */
#define SPEED 340 /* No prefix, no units */
#define x 82 /* Single letter — meaningless */
const int bankLimit = 30; /* camelCase, no prefix */
```

6. SECTION (d) — CODING CONDITIONS AND CONSTRAINTS

This section defines the conditions and constraints placed on HOW code is structured — specifically focusing on:

- The degree of coupling between software components (how much one module depends on another)
- The complexity of logical or numerical expressions (how hard an expression is to understand and test)
- The rationale for any permitted deviations from these constraints

Think of coupling like wiring in an aircraft. If every wire is connected to every other wire, a single short circuit could cause cascading failures across the whole aircraft. Software coupling works the same way: tightly-coupled modules make it impossible to isolate and test individual components.

NOTE *DO-178C requires that deviations from coding constraints be documented with justification. Simply writing complex or highly-coupled code without rationale is a finding during SOI-2 (Development Audit).*

6.1 Coupling Between Software Components

Coupling describes how much one software module depends on the internal details of another. Low coupling is required so that each module can be independently verified, tested, and updated without breaking other modules.

6.1.1 Permitted Coupling (Data Coupling): Module A passes data to Module B through a defined interface (function parameters or a data bus structure). This is the only permitted form of coupling.

6.1.2 Prohibited Coupling (Control Coupling): Module A tells Module B HOW to behave by passing control flags. This is prohibited because it creates hidden dependencies and makes structural coverage analysis unreliable.

6.1.3 Prohibited Coupling (Common Coupling): Two modules share the same global variable without a controlled interface. Prohibited except for explicitly designated global data stores (GDS) with access control functions.

AVIONICS EXAMPLE: Coupling — ADC to FMS Altitude Interface

```
/* PERMITTED: Data coupling — ADC publishes altitude via defined interface */
/* adc_data_bus.h — single controlled interface header */
typedef struct {
    float f32_baro_alt_ft; /* Barometric altitude, feet MSL */
    float f32_radio_alt_ft; /* Radio altitude, feet AGL */
    bool b_data_valid; /* Source validity flag */
} adc_output_t;

/* FMS only accesses ADC data through this function — never directly */
```

```
bool adc_get_altitude(adc_output_t* p_adc_out);

/* PROHIBITED: Common coupling – FMS directly reads ADC internal var */
extern float g_f32_raw_baro_ft; /* BANNED: direct global access */
fms_alt = g_f32_raw_baro_ft; /* BANNED: hidden dependency */
```

6.2 Complexity of Expressions – Cyclomatic Complexity

Cyclomatic Complexity (CC) is a measure of how many independent paths exist through a piece of code. Every if/else/while/for/switch adds one path. Higher complexity means more test cases are needed to achieve MC/DC structural coverage – and more chance of an undetected error.

CC Value	Risk Level	Action Required
1–10	Low Risk (GREEN)	Permitted – no action required
11–15	Medium Risk (YELLOW)	Permitted – senior reviewer must approve
16–20	High Risk (AMBER)	Deviation request required with DER approval
> 20	Unacceptable (RED)	PROHIBITED – function must be refactored

AVIONICS EXAMPLE: Cyclomatic Complexity – GPWS Mode 5 'Glideslope' Alert

```
/* CC = 1 (compliant) – simple, single-path function */
bool gpws_is_glideslope_alert_active(void) {
    return (g_f32_gs_deviation_dots > GPWS_GS_WARN_DOTS)
        && (g_b_gear_down == true)
        && (g_u16_radio_alt_ft < GPWS_GS_MAX_ALT_FT);
}

/* CC = 12 (marginal – requires senior review) */
/* Result: function was split into 3 sub-functions */
/* Each sub-function independently covers one mode */
```

6.3 Complexity of Logical and Numerical Expressions

Individual expressions within code shall be limited in complexity. Overly complex expressions make it impossible for a reviewer to determine correctness by inspection, and may produce incorrect results due to operator precedence surprises.

- Maximum Boolean terms per expression: 4 (before requiring extraction into intermediate variables)
- All non-obvious operator precedence shall be made explicit with parentheses
- No side effects in conditional expressions (no assignments inside if conditions)

- Numeric expressions involving unit conversions shall extract conversion factors to named constants

AVIONICS EXAMPLE: Expression Complexity — ILS Localizer Deviation

```
/* NON-COMPLIANT: 6 boolean terms, implicit precedence */
if (loc_dev > WARN && !inhibit && mode==ILS && alt<GS_ALT
    && vspd < 0 && gear_down) { ... }

/* COMPLIANT: Extracted intermediate Booleans, explicit parens */
bool b_below_gs_alt  = (f32_radio_alt_ft < ILS_GS_MAX_ALT_FT);
bool b_descending    = (f32_vspd_fpm < 0.0F);
bool b_ils_mode       = (e_ap_lat_mode == AP_LAT_ILS);
bool b_loc_deviation  = (f32_loc_dev_dots > ILS_LOC_WARN_DOTS);

if (b_loc_deviation && b_ils_mode && b_below_gs_alt
    && b_descending && b_gear_down && (!b_ils_alert_inhibit)) {
    gpws_set_alert(GPWS_ILS_LOC_CAUTION);
}
```

7. SECTION (e) – CONSTRAINTS ON THE USE OF CODING TOOLS

Coding tools are any software application used to write, compile, link, analyze, or generate the airborne software. DO-178C requires that all tools used in the software development process be either:

- Qualified under DO-178C Section 12 (Tool Qualification) as a Development Tool, OR
- Verified – meaning the tool's output is checked by another process that would detect any errors the tool introduces

Think of it this way: if a tool (like a compiler or code generator) could produce incorrect output that ends up in flight software, you must either prove the tool is correct (qualification) or check its output thoroughly (verification). You cannot simply trust that a tool works without evidence.

NOTE *Tool Operational Requirements (TOR) documents are required for each qualified tool. The TOR defines the tool's intended function, its limitations, and the conditions under which it is approved for use on this project.*

7.1 Approved Coding Tools

Tool / Version	Purpose	Tool Category	Qualification / Verification Approach
Green Hills MULTI v7.1.4	Compiler / Linker	DO-330 TQL-5	Output verified by object code review and disassembly inspection
LDRA Testbed v9.7	Static Analysis / MISRA	DO-330 TQL-4	Qualified; TOR-LDRA-001 on file; output checked by IV&V
VectorCAST 2024.1	Unit Test / Coverage	DO-330 TQL-1	Qualified; TOR-VCAST-002 on file; test results verified
IBM DOORS Next v7.0	Requirements Traceability	Not qualified	Output (traceability matrices) verified by manual review
Polyspace Bug Finder R2025b	Code Analysis	DO-330 TQL-4	Qualified; TOR-PSB-003; findings reviewed by IV&V engineer
Git v2.42 + GitLab CE	Configuration Management	Not a dev tool	CM tool – output verified by CM audit and baseline reports

7.2 Prohibited Tool Usage

- Code editors with auto-formatter features must have auto-formatting DISABLED (formatting must follow this SCSP, not the editor default)
- AI-assisted code generation tools (GitHub Copilot, ChatGPT, etc.) are PROHIBITED – generated code has no traceable design basis and cannot be verified to requirements
- Unapproved static analysis tools shall not be used – findings from non-qualified tools cannot be used as evidence in SOI audits

- Compiler optimization flags beyond -O1 are PROHIBITED for DAL A/B – higher optimization can eliminate code that affects structural coverage
- Debug macros (printf, fprintf to UART) must be controlled via conditional compilation and must be absent from the released build

AVIONICS EXAMPLE: Compiler Flags — Green Hills MULTI (DAL B Target: ARINC 653 RTOS)

APPROVED compiler invocation for DAL B flight software:

```
ccppc -cpu=ppc7457 \
-O1 \ /* Approved optimization level */
-misra2012 \ /* MISRA C:2012 enforcement */
-Wstrict \ /* All warnings as errors */
-DNDEBUG \ /* Removes assert() and debug */
-DBUILD_DAL_B \ /* DAL-level compilation guard */
-I./include \
-c src/fms_nav.c -o obj/fms_nav.o
```

PROHIBITED (DAL A/B):

```
ccppc -O3 -funroll-loops ... /* High optimization BANNED */
ccppc -DDEBUG_UART_ENABLE ... /* Debug output BANNED in release */
```

8. DO-178C COMPLIANCE MATRIX

The table below maps each DO-178C requirement relevant to Software Code Standards to the section of this document that satisfies it, and the evidence artifact generated.

DO-178C Reference	Requirement	SCSP Section	Evidence Artifact
Sec 11.1(a)	Programming language standards	Section 3	MISRA C compliance report, Language standard reference
Sec 11.1(b)	Code presentation standards	Section 4	Code review checklists (CRC), peer review records
Sec 11.1(c)	Naming conventions	Section 5	Static analysis reports (LDRA/Polyspace), code reviews
Sec 11.1(d)	Coding conditions and constraints	Section 6	Complexity metrics report, coupling analysis, CRC
Sec 11.1(e)	Constraints on use of coding tools	Section 7	Tool Qualification Records (TOR), TQL certificates
Sec 12	Tool qualification	Section 7.1	TOR documents, tool qualification plans (TQP)
Table A-5	Code review objectives (Obj 3–5)	All	Software Code Review Records (SCRR)

9. DEVIATION AND WAIVER PROCESS

A deviation is a planned departure from a requirement of this SCSP before the activity occurs. A waiver is approval of a departure after the activity has already occurred. Both require formal documentation and DER approval.

Example: If a legacy COTS module has a cyclomatic complexity of 22 (above the limit of 20), the project must file a Deviation Request (DR) before integrating that code. The DR must explain why the risk is acceptable and what compensating mitigations are in place.

Required Content of a Deviation Request (DR): Each DR shall include: (1) Identification of the specific SCSP requirement being deviated, (2) Rationale for the deviation, (3) Safety impact assessment, (4) Compensating mitigations (e.g., additional testing, formal methods, independent review), (5) DER/ACO approval signature, and (6) CM baseline reference.

NOTE *All approved deviations must be referenced in the Software Accomplishment Summary (SAS) submitted to the certification authority. Unapproved deviations found during SOI audits constitute a finding and may delay certification.*

10. GLOSSARY OF TERMS

Term	Plain-Language Definition
ACO	Aircraft Certification Office – the FAA office that approves the certification plan
CC (Cyclomatic)	A number measuring how many paths exist through a piece of code. Higher = harder to test
Coupling	How much one software module depends on another's internal workings
DAL	Design Assurance Level (A=most critical, E=least). Determines rigor of DO-178C activities
DER	Designated Engineering Representative – authorized to approve certification activities on behalf of FAA
DO-178C	The primary avionics software certification standard, published by RTCA in 2011
DO-330	The tool qualification standard – defines how to qualify a software development tool
GDS	Global Data Store – a controlled interface for inter-module data sharing
IV&V	Independent Verification and Validation – verification done by a team independent from development
LDRA	A static analysis and coverage tool commonly used in DO-178C projects
MC/DC	Modified Condition/Decision Coverage – the most stringent DO-178C coverage level (required for DAL A)
MISRA C	Motor Industry Software Reliability Association C coding standard, widely adopted in safety-critical systems
SCSP	Software Code Standards Plan – this document
SOI	Stage of Involvement – formal FAA/DER audit milestone (SOI-1=Planning, SOI-2=Development, SOI-3=Test, SOI-4=Final)
TOR	Tool Operational Requirements – document defining a tool's approved use for DO-178C
TQP	Tool Qualification Plan – document describing how a tool will be qualified under DO-330
VectorCAST	A unit testing and structural coverage analysis tool qualified for DO-178C projects

11. DOCUMENT CONTROL AND REVISION HISTORY

Rev	Date	Author	Summary of Changes
A	2026-05-04	IV&V Engineering	Initial release – all sections (a) through (e) established

END OF DOCUMENT

DO-178C Software Code Standards Plan – DOC-SCS-001 Rev A
This document is controlled. Verify latest revision before use.